TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

TRABAJO PRÁCTICO SEMANA 1

Gobernanza y Definición de Terminado (DoD)

Tecnicatura Universitaria en Programación a Distancia – UTN

Apellido y nombre:	Figueroa, Ignacio – DNI 45.406.120
Comisión:	C4 - A26

Proyecto usado

Para los tres trabajos usé un proyecto chico en Python que llamé **Tienda**: calcula precios y descuentos, cobra con una pasarela de pagos externa y genera reportes de ventas. El repositorio está organizado así:

```
tienda/
├── .github/
│   ├── ISSUE_TEMPLATE/reporte_de_bug.md   ← plantilla de incidencia (punto 2)
│   ├── labels.md                          ← sistema de etiquetas (punto 3)
│   └── pull_request_template.md
├── .githooks/pre-commit                   ← controles automáticos (TP semana 3)
├── docs/DOD.md                            ← Definition of Done (punto 1)
├── src/tienda/                            ← código del sistema
├── tests/                                 ← pruebas unitarias
└── pyproject.toml, .flake8                ← configuración de las herramientas
```

1. Definition of Done (DoD)

Una tarea se considera **terminada** solo cuando cumple los seis criterios. Si falta uno, el Pull Request no se integra a la rama principal (main).

Criterio	Qué se exige	Cómo se verifica
1. Integridad de compilación	El proyecto se instala e importa sin errores y sin warnings nuevos.	python -c "import tienda" y pipeline en verde.
2. Pruebas automatizadas	Todos los tests pasan (0 fallos). Cada funcionalidad nueva o bug corregido trae su test.	pytest, que corre en el pre-commit hook.
3. Cobertura mínima	Al menos 80% de cobertura de ramas. El código nuevo no puede bajar la cobertura.	pytest --cov. Falla si la cobertura es menor a 80%.
4. Revisión por pares	Al menos una aprobación de otro integrante. Nadie aprueba su propio código.	Regla de protección de la rama main en GitHub.
5. Gobernanza de estilo	Código formateado con black y sin errores de flake8. Complejidad ciclomática máxima de 10 por función.	black --check y flake8, en el pre-commit hook.
6. Documentación técnica	Funciones públicas con docstring. Si cambia el uso del sistema, se actualiza el README. El PR enlaza su issue con "Closes #N".	Revisión en el Pull Request.

Puse criterios que se puedan medir, como "80% de cobertura" o "complejidad máxima de 10", en lugar de cosas como "código de buena calidad", para que no se discuta si una tarea está terminada o no. Además, casi todos se verifican solos con herramientas, así no depende de que alguien se acuerde de revisarlo.

2. Plantilla de incidencia (Issue Template)

La guardé en .github/ISSUE_TEMPLATE/reporte_de_bug.md. Así GitHub la ofrece automáticamente al crear un issue nuevo.

.github/ISSUE_TEMPLATE/reporte_de_bug.md

```
---
name: Reporte de bug
about: Reportar un error del sistema
title: "[BUG] <módulo>: <qué falla en pocas palabras>"
labels: bug
assignees: ""
---

## Descripción técnica del problema
<!-- Qué falla, en qué módulo/función y desde cuándo. Ser específico. -->

## Pasos para reproducir
<!-- Pasos exactos y numerados. Otra persona tiene que poder repetirlo. -->
1.
2.
3.

## Comportamiento esperado
<!-- Qué debería pasar. -->

## Comportamiento observado
<!-- Qué pasa en realidad. Incluir el mensaje de error exacto. -->

## Entorno
- Versión / commit:
- Sistema operativo:
- Versión de Python:

## Evidencias
<!-- Capturas, logs o la salida de la consola. Pegar los logs entre ``` -->

## Prioridad
- [ ] Bloqueante
- [ ] Alta
- [ ] Media
- [ ] Baja

## Checklist DoD para cerrar este issue
- [ ] Hay un test que reproduce el bug y ahora pasa
- [ ] El PR enlaza este issue con `Closes #<número>`
- [ ] El PR tiene al menos una aprobación
```

Cada campo está pensado para que el que arregla el bug no tenga que volver a preguntar nada. El título tiene un formato fijo para identificar el módulo. Los pasos van numerados para que cualquiera pueda reproducir el error. Esperado y observado van separados para que quede claro dónde está la diferencia. Al final hay un checklist que conecta el issue con la DoD.

3. Matriz de categorización (etiquetas)

Propongo dos grupos de etiquetas: unas dicen **qué tipo de tarea** es y otras dicen **en qué estado está frente a la DoD**.

Tipo de tarea

Etiqueta	Cuándo se usa	Justificación: qué criterio de la DoD audita
bug	Algo que funciona mal o no hace lo que debería.	Pruebas (2): el PR tiene que traer un test que reproduzca el bug, así se comprueba que se arregló y que no vuelve a aparecer.
feature	Funcionalidad nueva.	Pruebas (2) y cobertura (3): el código nuevo viene con sus tests y no puede bajar la cobertura.
technical-debt	Código que funciona pero está mal hecho: duplicado, muy complejo o con dependencias viejas.	Estilo (5): la complejidad tiene que quedar en 10 o menos y las dependencias sin vulnerabilidades conocidas.
documentation	Cambios en el README, docstrings o estos documentos.	Documentación (6): permite ver qué tareas mantienen la documentación al día.

Estado frente a la DoD

Etiqueta	Significado
needs-review	El PR espera la revisión de otro integrante (criterio 4).
dod-blocked	Falla algún criterio de la DoD. No se puede integrar.
ready-to-merge	Cumple los 6 criterios y se puede integrar a main.

Con estas etiquetas, antes de integrar se puede filtrar en GitHub y ver rápido qué está bloqueado y qué está listo. Por ejemplo, buscar todos los PR con dod-blocked muestra qué falta resolver antes de una entrega.